

MACHINE LEARNING PROJECT BLUEPRINT

Design & Planning Document

Predicting Customer Churn for a Subscription Telecommunications Service

A supervised binary classification system for proactive customer retention

Field	Detail
Document Title	Project Blueprint — Customer Churn Prediction
Deliverable	Week 1: Project Blueprint Design and Planning
Version	1.0 (Planning Stage)
Project Duration	6 weeks / approx. 168 engineering hours
Primary Language & Stack	Python 3.11, scikit-learn, XGBoost, pandas, MLflow
Project Type	Supervised learning — binary classification
Document Status	For review and approval

Table of Contents

1. Executive Summary.....	3
2. Problem Statement and Objectives.....	4
2.1 Business Context.....	4
2.2 Problem Definition.....	4
2.3 Machine Learning Formulation.....	4
2.4 Objectives and Success Criteria.....	5
2.5 Scope.....	5
3. Data Requirements and Sources.....	7
3.1 Required Data Entities.....	7
3.2 Publicly Available Data Sources.....	7
3.3 Data Governance and Ethics.....	8
4. Machine Learning Pipeline Design.....	9
4.1 Stage 1 — Data Collection and Ingestion.....	9
4.2 Stage 2 — Data Preprocessing.....	9
4.3 Stage 3 — Exploratory Data Analysis.....	10
4.4 Stage 4 — Feature Engineering.....	11
4.5 Stage 5 — Model Building and Training.....	12
4.6 Stage 6 — Evaluation Strategy.....	12
4.7 Stage 7 — Deployment and Monitoring.....	13
5. Candidate Algorithms and Comparison.....	15
5.1 Algorithm 1 — Logistic Regression (L2 Regularised).....	15
5.2 Algorithm 2 — Random Forest.....	15
5.3 Algorithm 3 — XGBoost (Gradient Boosted Trees).....	16
5.4 Comparative Summary and Recommendation.....	16
6. Timeline and Resource Estimation.....	17
6.1 Effort Distribution.....	17
6.2 Milestones and Gates.....	17
6.3 Resource Requirements.....	18
7. Risk Analysis and Mitigation.....	19
7.1 Overall Risk Assessment.....	20
8. Project Deliverables and Structure.....	21
8.1 Expected Deliverables.....	21
8.2 Repository Structure.....	21
9. Conclusion.....	21
10. References.....	22

1. Executive Summary

This document presents the complete design and planning blueprint for a machine learning project intended to predict customer churn for a subscription-based telecommunications provider. It is written at the planning stage: no model has been built yet, and the purpose of the document is to establish a defensible plan before any code is written.

The project treats churn prediction as a supervised binary classification problem. For every active customer, the system estimates the probability that the customer will cancel their service within the next 30 days. Customers scored above a business-calibrated threshold are routed to the retention team, which intervenes with a targeted offer. The value of the system is therefore not measured by classification accuracy alone, but by the net revenue retained after subtracting the cost of the retention offers.

The blueprint below defines the problem and its success criteria, specifies the data required and the publicly available datasets that will be used to build the first version, designs the full machine learning pipeline from ingestion to monitoring, compares three candidate algorithms with a recommendation, lays out a six-week timeline with hour-level estimates, and closes with a risk register and mitigation strategy.

Headline recommendation: Build a regularised Logistic Regression as the interpretable baseline, then a Gradient Boosted Tree (XGBoost) as the production candidate. Promote the boosted model only if it beats the baseline by a meaningful margin on PR-AUC and on the business cost metric, because the baseline is materially cheaper to explain, deploy, and maintain.

2. Problem Statement and Objectives

2.1 Business Context

The hypothetical client, referred to throughout as NorthLink Communications, is a mid-sized telecommunications provider serving approximately 400,000 residential subscribers across broadband, mobile, and bundled television plans. Revenue is recurring and monthly. Growth in the provider's market has flattened, which means revenue is now defended primarily by keeping existing customers rather than by acquiring new ones.

NorthLink currently loses roughly 2% of its subscriber base each month, an annualised churn rate in the region of 22%. The retention team is aware of churn only after a cancellation request arrives through the call centre or the self-service portal. At that point the customer has already decided to leave, and intervention consists of a discount offered under pressure, which succeeds infrequently and erodes margin when it does succeed.

Acquiring a replacement subscriber costs the business several times more than retaining an existing one. The commercial opportunity is therefore to move the point of intervention earlier: to identify customers who are likely to churn while they are still undecided, and to contact them before they initiate a cancellation.

2.2 Problem Definition

The retention team cannot act early because it has no reliable way to rank customers by their likelihood of leaving. Its current approach uses a small set of manually written rules, such as flagging any customer whose contract expires within 60 days or who has logged more than two support complaints in a month. These rules are coarse. They flag large numbers of customers who were never going to leave, and they miss customers who churn for reasons the rules do not encode, such as a gradual decline in data usage or a competitor entering their service area.

The problem to be solved is therefore one of prioritisation under a fixed operational budget. The retention team can contact a limited number of customers per week. The system must tell them which customers those should be.

2.3 Machine Learning Formulation

The problem maps cleanly onto supervised binary classification. Each row in the modelling dataset is one customer observed at one point in time, described by features drawn strictly from the period before that point. The label is whether that customer cancelled their service during the subsequent observation window.

Element	Specification
Learning task	Supervised binary classification
Unit of observation	One customer at one monthly snapshot date
Target variable	churn = 1 if the customer cancels within 30 days of the snapshot date; otherwise churn = 0
Model output	A calibrated probability in the range 0 to 1, not a hard class label
Feature window	Behaviour and account data from the 12 months preceding the snapshot date
Prediction window	The 30 days following the snapshot date
Expected class balance	Approximately 20–27% positive class, based on published telecom churn datasets

Element	Specification
Refresh cadence	Batch scoring of the full active customer base, weekly

Table 2.1 — Formal specification of the learning task.

Two design choices in the table above deserve emphasis. First, the model outputs a probability rather than a label. This matters because the retention team's capacity changes week to week; a probability lets the business move the cut-off up or down without retraining. Second, features are restricted to the period strictly before the snapshot date. This constraint is what prevents data leakage, which is discussed in detail in Section 4.2 and appears as the highest-severity item in the risk register.

2.4 Objectives and Success Criteria

Success is defined on three levels. Technical criteria confirm that the model works; business criteria confirm that the model is worth deploying; operational criteria confirm that it can be maintained.

Level	Criterion	Target	Rationale
Technical	PR-AUC (average precision)	≥ 0.55	Robust to class imbalance; measures ranking quality, which is what the retention team actually consumes
Technical	Recall at the top 10% of scored customers	≥ 0.45	Captures at least 45% of all churners within the contactable budget
Technical	Precision at the top 10%	≥ 0.50	At most half of retention contacts are wasted on non-churners
Technical	Calibration (Brier score)	≤ 0.12	Probabilities must be trustworthy because they feed an expected-value calculation
Business	Net revenue retained per quarter	> cost of offers + project cost	The model must pay for itself; this is the decisive metric
Business	Uplift vs. current rule-based process	$\geq 25\%$ more churners identified at equal contact volume	Establishes that the model beats the incumbent, not just a random baseline
Operational	Batch scoring runtime	< 30 minutes for 400k customers	Must complete inside the weekly operations window
Operational	Explainability	Top 5 contributing factors per flagged customer	Retention agents need a reason to open the conversation with

Table 2.2 — Success criteria across technical, business, and operational levels.

Classification accuracy is deliberately excluded from this table. With a churn rate near 25%, a model that predicts "no churn" for every customer achieves 75% accuracy while being completely useless. Accuracy will be reported for completeness but will never be used to select a model.

2.5 Scope

In Scope

- Voluntary churn by residential subscribers — customers who actively choose to cancel.
- A batch scoring pipeline producing a weekly ranked list of at-risk customers.

- Per-customer explanations of the drivers behind each score.
- A documented retraining and monitoring procedure.

Out of Scope

- Involuntary churn such as disconnection for non-payment, which is a credit and collections problem with different drivers and a different owner.
- Business and enterprise accounts, whose contract structures and churn dynamics differ substantially from residential ones.
- Prescribing which retention offer to make. The model ranks risk; choosing the intervention is a separate uplift-modelling problem noted as future work.
- Real-time or streaming inference. Weekly batch scoring is sufficient for a 30-day prediction window and avoids substantial infrastructure cost.

3. Data Requirements and Sources

3.1 Required Data Entities

The features that predict churn fall into five families. The table below lists each family, the attributes it contributes, and the reason it is expected to carry signal. This specification is written independently of any particular dataset so that it can also serve as the data request issued to the client's data engineering team when the project moves onto real production data.

Data Family	Representative Attributes	Why It Is Expected to Predict Churn
Demographic and account profile	Tenure in months, senior citizen flag, presence of partner or dependents, geographic region	Tenure is consistently the single strongest churn predictor; risk is heavily front-loaded in the first year. Household composition proxies for switching friction.
Subscription and contract	Contract type (month-to-month, one-year, two-year), payment method, paperless billing flag, auto-renewal status	A month-to-month contract removes the contractual barrier to leaving. Payment method is a well-documented proxy for engagement and financial stability.
Service portfolio	Internet service type, phone lines, streaming TV, technical support add-on, online security add-on	Each additional bundled service raises the effort required to switch provider. Customers holding a single service churn at markedly higher rates.
Usage and engagement	Monthly data volume, call minutes, login frequency, month-over-month change in usage, days since last app session	A sustained decline in usage frequently precedes cancellation by several weeks, which makes this family the most valuable source of early warning signal.
Billing and service experience	Monthly charges, total charges, recent price change, number of support tickets, unresolved ticket count, outage minutes experienced	Price increases and unresolved service failures are the two most common stated reasons for leaving in customer exit surveys.

Table 3.1 — Required data families and their expected predictive contribution.

Two additional requirements apply across all five families. Every attribute must carry a valid-from timestamp so that features can be reconstructed as they stood on any historical snapshot date; without this, point-in-time correctness cannot be guaranteed. And the labelled history must span at least 24 months so that the model is exposed to a full cycle of seasonal effects and at least one competitive promotional period.

3.2 Publicly Available Data Sources

Because this project is executed as a portfolio exercise rather than against a live production warehouse, the first iteration will be built on publicly available datasets. The selection below is ordered by intended use: one primary dataset to establish the end-to-end pipeline, and three supporting datasets to stress-test the design against larger volumes and richer behavioural signal.

Dataset	Source & Access	Size and Content	Justification for Selection
IBM Telco Customer Churn	Kaggle / IBM Community — open, CSV download, no licence restriction for educational use	≈7,043 customers, 21 attributes, ≈26.5% churn rate	PRIMARY. Its schema maps almost one-to-one onto the demographic, contract, service, and billing families in Table 3.1, and it is the de facto benchmark for telecom churn, so results are comparable against widely published work.

Dataset	Source & Access	Size and Content	Justification for Selection
Cell2Cell Telecom Churn	Duke University Teradata Center / Kaggle mirror	≈71,000 customers, 58 attributes, heavy behavioural and call-detail features	Provides the usage and engagement family that the primary dataset lacks, and a volume large enough to make feature engineering and training cost realistic rather than instantaneous.
Telco Customer Churn (Orange / KDD Cup 2009)	KDD Cup 2009 archive, publicly released	50,000 customers, 230 anonymised predictors, severe class imbalance	A deliberate stress test. Its anonymised columns and 7% churn rate force the pipeline to cope with high dimensionality and extreme imbalance rather than a conveniently clean dataset.
UCI Bank Marketing / Churn Modelling	UCI Machine Learning Repository and Kaggle	10,000–45,000 records, tabular, binary outcome	Used only as a methodological cross-check to confirm that the pipeline generalises to a different subscription domain and is not overfitted to telecom-specific quirks.

Table 3.2 — Candidate public datasets, with the role each plays in the project.

Why These Sources Are Suitable

Four properties made these datasets the right choice. Each is genuinely labelled, meaning the churn outcome is recorded rather than inferred, which is a prerequisite for supervised learning and is surprisingly rare in freely available telecom data. Each is openly licensed for educational and research use, so the project carries no redistribution risk. Their class balance sits in the realistic 7–27% band rather than being artificially balanced, which preserves the imbalance problem the production system will actually face. And taken together they span the schema described in Table 3.1, with the primary dataset supplying contract and billing attributes and the Cell2Cell data supplying behavioural attributes.

Acknowledged Limitations of the Public Data

The public datasets are a reasonable proxy but not a substitute for production data, and the plan is explicit about the gap. The IBM dataset is cross-sectional: it captures a single snapshot per customer with no timestamps, which means true temporal validation is impossible on it and the model cannot learn trend features such as declining usage. It is also small enough that a complex model will overfit without aggressive regularisation. None of the datasets contains the free-text support-ticket content that in practice often carries the clearest churn signal. These limitations are carried forward into the risk register in Section 7 as R-04.

3.3 Data Governance and Ethics

Even working with public data, the project will follow the practices that a production deployment would require, since the code and design are intended to transfer. Direct identifiers such as customer ID are excluded from the feature set and retained only as join keys. Protected attributes — in this data, the senior citizen flag, partner status, and dependent status — will be examined explicitly for disparate impact during evaluation, comparing false positive and false negative rates across groups. This matters because a retention model that systematically under-flags a particular group effectively withholds discounts from that group. Any group disparity above five percentage points will be treated as a blocking issue requiring mitigation before deployment.

4. Machine Learning Pipeline Design

The pipeline is designed as seven sequential stages. Each stage has a defined input, a defined output, and an acceptance check that must pass before the next stage begins. Every stage is implemented as a versioned, parameterised module so that the entire pipeline can be re-executed end to end from raw data with a single command — a property that matters far more for reproducibility than any individual modelling decision.

Stage	Input	Output	Acceptance Check
1. Data collection & ingestion	Raw CSV sources	Versioned raw table in Parquet	Row and column counts match source; schema contract validated
2. Preprocessing & cleaning	Raw table	Clean, typed analytical table	Zero unexpected nulls; no duplicate customer-date keys
3. Exploratory data analysis	Clean table	EDA notebook and findings log	Every feature family in Table 3.1 profiled and reviewed
4. Feature engineering	Clean table	Feature matrix + feature dictionary	Point-in-time correctness verified; no leakage-flagged feature present
5. Model building & training	Feature matrix	Trained, serialised candidate models	All experiments logged to MLflow with seeds and parameters
6. Evaluation & selection	Candidate models	Selection report, one chosen model	Held-out test set touched exactly once
7. Deployment & monitoring	Chosen model	Scoring job + monitoring dashboard	End-to-end dry run completes inside the operations window

Table 4.1 — Pipeline stages with inputs, outputs, and gating checks.

4.1 Stage 1 — Data Collection and Ingestion

Raw files are downloaded once and stored immutably. Nothing downstream ever writes back to the raw layer; every transformation produces a new artefact. Each ingested file is hashed and recorded alongside its source URL and retrieval date, so that any result can be traced back to the exact bytes that produced it.

Ingestion applies a schema contract rather than trusting the file. Expected column names, data types, and permitted value ranges are declared in a configuration file, and any deviation halts the pipeline with an explicit error instead of silently coercing a type. This single practice prevents the most common category of production data incident, in which an upstream schema change propagates quietly into the model.

Tooling: pandas and PyArrow for ingestion, Parquet for storage, Pandera for schema validation, DVC for data versioning.

4.2 Stage 2 — Data Preprocessing

Preprocessing is expressed entirely as a scikit-learn Pipeline object rather than as notebook cells. This is a deliberate architectural decision: every transformation learns its parameters from the training fold alone and then applies them to validation and test folds, which structurally eliminates the preprocessing leakage that otherwise inflates cross-validation scores.

Step	Approach	Design Notes
Type correction	Coerce numeric columns stored as text; parse dates	In the IBM dataset TotalCharges is stored as text and contains blanks for tenure-zero customers — a known trap that silently produces nulls
Duplicate handling	Deduplicate on the customer-plus-snapshot-date key	Exact duplicates are dropped; near-duplicates are logged for review rather than removed automatically
Missing values	Median imputation for numeric, explicit 'Unknown' category for categorical, with a companion missingness indicator	Missingness is often informative — a blank tenure charge means a brand-new customer — so the indicator preserves that signal instead of discarding it
Outlier treatment	Winsorise continuous variables at the 1st and 99th percentiles	Applied only to linear models; tree-based models are left untreated because they are insensitive to monotone outliers
Categorical encoding	One-hot for low-cardinality; target encoding computed inside cross-validation folds for high-cardinality	Target encoding computed on the full dataset is a classic leakage source and is explicitly forbidden here
Scaling	Standardisation for Logistic Regression only	Required for regularised linear models; unnecessary and unhelpful for trees
Class imbalance	Class weighting first; SMOTE evaluated as an alternative but applied only to training folds	Weighting is preferred because it preserves calibration, whereas resampling distorts predicted probabilities and this project needs trustworthy probabilities

Table 4.2 — Preprocessing steps and the reasoning behind each choice.

Preventing Data Leakage

Leakage is the single most likely cause of this project producing an excellent offline score and a worthless deployed model, so it is controlled at three separate points. First, the feature window is bounded: only data with a timestamp strictly earlier than the snapshot date may enter a feature. Second, a leakage audit is run over the candidate feature list before training, in which any feature with suspiciously high univariate association with the target is manually inspected — fields such as cancellation reason code, final bill amount, or disconnection date are artefacts of the outcome and must be removed. Third, all fitted transformations live inside the cross-validation loop, never outside it.

4.3 Stage 3 — Exploratory Data Analysis

EDA is treated as a hypothesis-generating stage with defined outputs rather than as open-ended exploration, because unbounded EDA is where project timelines most often slip.

- Univariate profiling: distribution, missingness rate, and cardinality for every column, produced as a standard profile report.
- Target relationship analysis: churn rate broken down by each categorical level and by decile of each continuous variable, which reveals non-linear relationships that inform feature engineering.
- Tenure and survival analysis: a Kaplan-Meier curve of survival against tenure, to locate the periods of elevated risk that justify the tenure-band feature.
- Correlation and multicollinearity screening: a correlation matrix plus variance inflation factors, since monthly charges, total charges, and tenure are mathematically entangled and will destabilise a linear model.

- Segment analysis: churn rate cross-tabulated by contract type against service portfolio, to identify the high-risk segments the business will care about most.

Stage output: a written findings log of confirmed and refuted hypotheses, which becomes the direct input to feature engineering. EDA is time-boxed to 16 hours.

4.4 Stage 4 — Feature Engineering

Feature engineering is expected to contribute more to model performance than algorithm selection does. The features below are derived rather than raw, and each is motivated by a specific hypothesis about churn behaviour.

Engineered Feature	Definition	Hypothesis
tenure_band	Tenure bucketed into 0–6, 7–12, 13–24, 25–48, 49+ months	Churn risk is non-linear in tenure, spiking early and flattening later; banding lets a linear model capture this
avg_charge_per_month	Total charges divided by tenure	Detects customers whose current price has drifted above their historical average, a common trigger for switching
price_to_segment_ratio	Monthly charge divided by the median charge of customers with the same service portfolio	Identifies customers paying above the going rate for what they receive
service_count	Count of active add-on services	Each additional service increases switching friction; a direct measure of account stickiness
usage_trend_3m	Slope of monthly usage over the trailing three months	A sustained decline in usage is the clearest early warning of disengagement
usage_vs_own_baseline	Current month usage divided by the customer's own 12-month median	Normalises against individual habits, so a light user who halves their usage is flagged as strongly as a heavy user who does
support_intensity_90d	Support tickets raised in the last 90 days, scaled by tenure	Repeated contact indicates unresolved friction; scaling prevents long-tenured customers from dominating the count
unresolved_ticket_flag	Binary flag for any open ticket older than 14 days	An unresolved complaint is a concrete, actionable grievance
contract_days_remaining	Days until contract expiry	Churn clusters sharply around renewal decision points
recent_price_increase	Binary flag for a charge increase above 5% in the last 90 days	Price change is among the most frequently cited reasons for cancellation

Table 4.3 — Engineered feature specification with supporting hypotheses.

Features in this table that depend on time series data — the usage trend and baseline ratio features in particular — cannot be constructed from the cross-sectional IBM dataset. They will be built and validated on the Cell2Cell data, and the feature dictionary records which dataset supports which feature so the limitation stays visible rather than being quietly forgotten.

Feature selection then proceeds in three passes: drop near-zero-variance columns, remove one member of any pair correlated above 0.90 while keeping the more interpretable of the two, and rank the remainder by permutation importance measured on a validation fold. Recursive feature

elimination is applied only if the final feature count exceeds roughly 60, since beyond that point marginal features tend to add variance rather than signal.

4.5 Stage 5 — Model Building and Training

Data Splitting Strategy

Where timestamps are available, splits are temporal rather than random: the earliest 70% of snapshots form the training set, the next 15% the validation set, and the most recent 15% the test set. A random split would allow the model to learn from the future to predict the past, which inflates scores and misrepresents deployed performance. Where the dataset is cross-sectional and temporal splitting is impossible, a stratified random split is used and the limitation is reported explicitly alongside the results.

Model selection and hyperparameter tuning use five-fold stratified cross-validation on the training portion. The test set is held back and evaluated exactly once, after the final model has been chosen. Any subsequent decision informed by the test set would convert it into a validation set and destroy the unbiased estimate it exists to provide.

Split	Proportion	Purpose
Training	70%	Fitting model parameters and all preprocessing transformations
Validation	15%	Hyperparameter search, threshold calibration, early stopping
Test	15%	Single final unbiased performance estimate

Table 4.4 — Data splitting strategy.

Training Protocol

1. Establish a baseline: the client's existing rule set, reimplemented as code, so that every model is measured against the process it would replace rather than against random guessing.
2. Train Logistic Regression with L2 regularisation as the interpretable reference model.
3. Train Random Forest and XGBoost as the non-linear candidates.
4. Tune hyperparameters using Bayesian optimisation via Optuna, with 50 trials per model, scoring on cross-validated PR-AUC.
5. Calibrate the selected model's probabilities using isotonic regression on the validation set, then verify with a reliability diagram.
6. Log every run to MLflow with its random seed, parameters, metrics, and a hash of the training data, so any result can be reproduced exactly.

4.6 Stage 6 — Evaluation Strategy

Evaluation proceeds on three fronts, and a model must satisfy all three before it is considered deployable.

Statistical Evaluation

Metric	Why It Is Used	Target
PR-AUC / Average Precision	Primary selection metric; unlike ROC-AUC it is not flattered by a large, easily classified negative class	≥ 0.55
Precision@k and Recall@k	Mirrors the real operating constraint: a fixed weekly contact	≥ 0.50 and ≥ 0.45

Metric	Why It Is Used	Target
(k = top 10%)	budget	
ROC-AUC	Reported for comparability with published benchmarks on these datasets	≥ 0.84
F1 score at the chosen threshold	Single-number summary once the operating point is fixed	Reported, not optimised
Brier score and reliability curve	Confirms probabilities are calibrated, which the expected-value calculation depends on	≤ 0.12
Lift chart by decile	The format the business already reads; shows concentration of churners in the top deciles	Top decile lift $\geq 2.5\times$

Table 4.5 — Evaluation metrics, purpose, and acceptance targets.

Business Evaluation

The statistical metrics are converted into currency through a cost matrix, because the two error types are not equally expensive. A false negative loses a customer's remaining lifetime value. A false positive costs only the retention offer extended unnecessarily, and only where the offer is accepted. Since lifetime value substantially exceeds offer cost, the optimal decision threshold sits well below 0.5, and it is derived from this matrix rather than assumed.

Outcome	Interpretation	Financial Consequence
True positive	Churner correctly identified and contacted	Offer cost incurred; a share of these customers are retained, recovering their remaining lifetime value
False positive	Loyal customer incorrectly flagged	Cost of an unnecessary discount, plus minor margin erosion
False negative	Churner missed entirely	Full loss of remaining customer lifetime value — the most expensive error by a wide margin
True negative	Loyal customer correctly left alone	No cost

Table 4.6 — Cost matrix used to derive the operating threshold.

Robustness and Fairness Evaluation

- Segment-level performance: metrics recomputed separately for each contract type, tenure band, and service tier, to confirm the model is not carried entirely by one easy segment.
- Fairness audit: false positive and false negative rates compared across the protected attributes identified in Section 3.3, with a five-percentage-point disparity treated as a blocking issue.
- Stability check: the model retrained on shifted time windows to confirm that performance and feature importances do not swing wildly between periods.
- Explainability: SHAP values computed globally to validate that the model's learned relationships are directionally sensible, and locally to supply the top five drivers for each flagged customer.

4.7 Stage 7 — Deployment and Monitoring

The deployment target is a scheduled weekly batch job that scores the active customer base and writes a ranked table, with scores and explanations, to the retention team's operational system. Batch scoring is chosen over a real-time API because a 30-day prediction window makes sub-second latency pointless, and the simpler architecture removes an entire class of operational risk.

Monitoring covers four signals: input drift measured by population stability index on each feature, prediction drift in the distribution of scores, performance decay tracked as actual churn outcomes arrive after each 30-day window closes, and pipeline health in the form of runtime and failure alerts. A PSI above 0.25 on any important feature, or a PR-AUC decline exceeding ten percent from the baseline, triggers investigation and probable retraining. Absent any trigger, the model is retrained quarterly on a rolling window.

5. Candidate Algorithms and Comparison

Three algorithms are proposed. Logistic Regression serves as the interpretable reference, Random Forest as a robust non-linear alternative requiring little tuning, and XGBoost as the highest-ceiling candidate. All three will be trained and compared on identical features and folds; the comparison below sets expectations and records the reasoning in advance so that the final choice is a decision rather than a rationalisation.

5.1 Algorithm 1 — Logistic Regression (L2 Regularised)

A linear model estimating the log-odds of churn as a weighted sum of features. It is the natural baseline for tabular binary classification and remains the most widely deployed churn model in industry, largely because its coefficients translate directly into statements a business stakeholder can act on.

Advantages	Limitations
Coefficients are directly interpretable as odds ratios, which satisfies the explainability requirement without additional tooling	Cannot capture non-linear relationships or interactions unless they are engineered in by hand
Produces well-calibrated probabilities natively, which the expected-value threshold calculation depends on	Sensitive to multicollinearity, which is a live concern given the tenure, monthly charge, and total charge relationship
Trains in seconds and scores 400,000 customers almost instantly	Typically underperforms tree ensembles on tabular data by a few points of PR-AUC
Low variance, so it performs comparatively well on the small primary dataset	Requires scaling and careful encoding, adding preprocessing steps that trees do not need
Trivial to deploy, monitor, and explain to a regulator or an auditor	Outlier-sensitive, requiring the winsorisation step in Table 4.2

Table 5.1 — Logistic Regression: advantages and limitations.

5.2 Algorithm 2 — Random Forest

An ensemble of decision trees trained on bootstrapped samples with randomised feature subsets, aggregated by majority vote. It captures non-linearity and interactions automatically and is notably forgiving of poor hyperparameter choices.

Advantages	Limitations
Captures non-linear effects and feature interactions with no manual specification	Substantially larger model artefact and slower scoring than a linear model
Requires no feature scaling and tolerates outliers and mixed data types	Individual predictions are not directly interpretable; explanation requires SHAP as an additional layer
Bagging makes it resistant to overfitting and stable across reruns	Default probability outputs are poorly calibrated and tend to cluster toward the middle of the range
Performs reliably with minimal tuning, which makes it a dependable second reference point	Can be biased toward high-cardinality categorical features when using impurity-based importance
Provides out-of-bag error estimates and usable feature importances essentially for free	Generally a point or two of PR-AUC behind a well-tuned boosted ensemble on this class of problem

Table 5.2 — Random Forest: advantages and limitations.

5.3 Algorithm 3 — XGBoost (Gradient Boosted Trees)

Trees fitted sequentially, each correcting the residual errors of its predecessors, with explicit regularisation on tree complexity. Gradient boosted ensembles are the consistent state of the art for tabular classification and are the expected top performer here.

Advantages	Limitations
Consistently the strongest performer on structured tabular data, and the likely winner on PR-AUC	The most hyperparameter-sensitive of the three; poor settings overfit quickly, especially on the small primary dataset
Native handling of missing values and of class imbalance through the <code>scale_pos_weight</code> parameter	Requires a validation set and early stopping to avoid overfitting, consuming data that could otherwise train the model
Built-in L1 and L2 regularisation give direct control over model complexity	Training cost is materially higher, and the Optuna search is the single largest compute line item in the project
Integrates cleanly with SHAP, giving fast and exact tree-based explanations	Sequential training makes it harder to parallelise than a Random Forest
Mature, well-supported, and straightforward to serialise and deploy	Probabilities require post-hoc calibration before they can be used in the cost calculation

Table 5.3 — XGBoost: advantages and limitations.

5.4 Comparative Summary and Recommendation

Criterion	Logistic Regression	Random Forest	XGBoost
Expected PR-AUC	Moderate (baseline)	Good	Best
Interpretability	High — native	Low — needs SHAP	Low — needs SHAP
Probability calibration	Strong out of the box	Weak	Needs calibration
Tuning burden	Minimal	Low	High
Training cost	Seconds	Minutes	Tens of minutes with search
Overfitting risk on small data	Low	Low to moderate	High without regularisation
Maintenance burden	Low	Moderate	Moderate to high

Table 5.4 — Side-by-side comparison of the three candidates.

The recommendation is to train all three and adopt a two-model outcome. Logistic Regression is retained permanently as the interpretable reference and as the fallback should the production model fail. XGBoost is promoted to production only if it exceeds the Logistic Regression baseline by at least three points of PR-AUC and also wins on the net revenue metric in Section 4.6. If the margin is smaller than that, the simpler model is deployed, on the grounds that a marginal accuracy gain does not justify the additional tuning, calibration, explanation, and monitoring burden that the boosted model carries. Random Forest serves as the tie-breaker: if it closely matches XGBoost while needing far less tuning, it becomes the preferred non-linear option.

6. Timeline and Resource Estimation

The project is planned across six calendar weeks at approximately 28 hours per week, totalling 168 hours. Estimates are given per phase and include a contingency allowance, because data cleaning in particular reliably consumes more time than first estimated. The plan assumes a single practitioner with part-time access to a domain expert for requirements review and result validation.

Phase	Key Activities	Est. Hours	Week
1. Problem definition & planning	Stakeholder framing, success criteria, scope, this blueprint document	18	1
2. Data acquisition & understanding	Source evaluation, download, schema contracts, data dictionary	14	1–2
3. Exploratory data analysis	Univariate and bivariate profiling, survival analysis, correlation screening, findings log	16	2
4. Data preprocessing	Cleaning, imputation, encoding, imbalance handling, pipeline construction	24	2–3
5. Feature engineering	Deriving the features in Table 4.3, leakage audit, feature selection, feature dictionary	22	3
6. Model building & training	Baseline reimplementations, three candidate models, Optuna tuning, MLflow tracking	28	4
7. Evaluation & selection	Metric computation, threshold optimisation, cost analysis, fairness and segment audits, SHAP	20	4–5
8. Deployment design & documentation	Scoring job, monitoring plan, retraining procedure, final report and presentation	16	5–6
9. Contingency & review buffer	Rework from review feedback, unforeseen data issues	10	6
TOTAL		168	6 weeks

Table 6.1 — Phase-level timeline with estimated effort.

6.1 Effort Distribution

Roughly 45% of the total effort falls in data acquisition, exploration, preprocessing, and feature engineering, against 29% in model building and evaluation. This distribution is intentional and matches the consistent industry observation that data preparation dominates machine learning projects. A plan that allocated most of its hours to modelling would be a plan that had not anticipated reality.

6.2 Milestones and Gates

Milestone	End of	Gate Criterion
M1 — Blueprint approved	Week 1	Problem statement, success criteria, and data sources signed off
M2 — Analysis-ready dataset	Week 3	Clean feature matrix produced; leakage audit passed and documented
M3 — Candidate models trained	Week 4	All three algorithms trained, tuned, and logged to MLflow
M4 — Model selected	Week 5	Test set evaluated once; selection justified on statistical and business metrics

Milestone	End of	Gate Criterion
M5 — Project handover	Week 6	Deployment and monitoring plan documented; final report delivered

Table 6.2 — Milestones and the criteria required to pass each gate.

6.3 Resource Requirements

Category	Requirement	Notes
Personnel	1 ML practitioner (full allocation); domain expert (≈ 4 hours total)	Domain expert time is concentrated at M1 and M4, for requirements review and result validation
Compute	16 GB RAM, 4+ CPU cores; GPU not required	Datasets are small enough for in-memory processing; gradient boosting on 71k rows is CPU-feasible
Software	Python 3.11, pandas, scikit-learn, XGBoost, imbalanced-learn, SHAP, Optuna, MLflow, matplotlib, seaborn	All open-source, so there is no licensing cost
Storage	≈ 5 GB for raw data, intermediate artefacts, and model versions	Parquet for tabular artefacts; DVC for versioning
Environment	Git repository, virtual environment with pinned dependencies, Jupyter for exploration	Pinned dependencies are a reproducibility requirement, not a convenience

Table 6.3 — Resource requirements.

7. Risk Analysis and Mitigation

Risks are rated on likelihood and impact, each on a three-point scale, and ordered by severity. Each risk has a named mitigation to reduce its probability and a contingency describing the response if it materialises anyway. The first two entries are the ones most likely to determine whether this project succeeds.

ID	Risk	L / I	Mitigation and Contingency
R-01	Data leakage inflates offline performance, producing a model that scores excellently in development and fails in production	Med / High	Mitigation: enforce a strict pre-snapshot feature window; run a mandatory leakage audit before training; fit all transformations inside cross-validation folds. Contingency: if a held-out temporal evaluation diverges sharply from cross-validation, halt and re-audit the feature list before proceeding.
R-02	Class imbalance causes the model to favour the majority class and under-identify churners	High / High	Mitigation: select on PR-AUC rather than accuracy; apply class weighting; derive the threshold from the cost matrix rather than defaulting to 0.5. Contingency: evaluate SMOTE on training folds only, and if used, recalibrate probabilities afterwards.
R-03	Concept drift — customer behaviour or the competitive landscape shifts and model performance decays after deployment	High / Med	Mitigation: monitor PSI on inputs and scores; track realised performance as outcomes arrive; retrain quarterly by default. Contingency: trigger immediate retraining on a PSI breach above 0.25 or a PR-AUC decline exceeding 10%.
R-04	Public datasets are not representative of the production environment, limiting transferability	High / Med	Mitigation: state the limitation explicitly; validate the methodology across multiple datasets; keep the pipeline schema-driven rather than dataset-specific. Contingency: treat the result as a validated methodology and a performance range rather than as a production-ready model.
R-05	Poor data quality — missing values, inconsistent types, or undocumented fields consume more time than allocated	High / Med	Mitigation: enforce schema contracts at ingestion; time-box cleaning; profile early in Week 2. Contingency: drop features whose quality cannot be resolved inside the time-box and record the exclusion in the feature dictionary.
R-06	Overfitting, particularly likely for XGBoost on the ≈7,000-row primary dataset	Med / High	Mitigation: regularisation, early stopping on a validation fold, cross-validated tuning, and a deliberate preference for the simpler model at small performance margins. Contingency: fall back to the Logistic Regression baseline, which is retained for exactly this purpose.
R-07	The model shows disparate performance across protected groups	Med / High	Mitigation: audit false positive and false negative rates by group as a standard evaluation step; exclude protected attributes from features while retaining them for auditing. Contingency: treat any disparity above 5 percentage points as blocking; apply threshold adjustment or reweighting before deployment.
R-08	Low stakeholder adoption because retention agents do not trust or understand the scores	Med / High	Mitigation: deliver the top five SHAP drivers alongside each score; involve the domain expert at M1 and M4; benchmark against the incumbent rule set so the improvement is visible in familiar terms. Contingency: run a shadow period where the model and the existing rules operate in parallel before

ID	Risk	L / I	Mitigation and Contingency
			switching over.
R-09	Scope creep — requests for uplift modelling, offer optimisation, or real-time scoring	Med / Med	Mitigation: the explicit out-of-scope list in Section 2.5; formal change control at milestone reviews. Contingency: log new requests as a phase two backlog rather than absorbing them into the current timeline.
R-10	Timeline slippage, most likely originating in preprocessing	Med / Med	Mitigation: 10 hours of contingency built into Phase 9; time-boxed EDA and cleaning; milestone gates that surface slippage early. Contingency: reduce scope to the primary dataset and the two strongest algorithms, preserving the end-to-end pipeline over breadth of experimentation.

Table 7.1 — Risk register. L = likelihood, I = impact.

7.1 Overall Risk Assessment

The dominant risks are methodological rather than technical. Leakage and imbalance mishandling are the two failure modes most likely to produce a model that looks successful and is not, and both are addressed structurally — by pipeline architecture and by metric selection — rather than by relying on vigilance. The residual risk after mitigation is judged acceptable for a project of this scope, with the representativeness limitation in R-04 being the constraint that most affects how the final results should be interpreted.

8. Project Deliverables and Structure

8.1 Expected Deliverables

Deliverable	Format	Due
Project blueprint (this document)	DOCX / PDF	Week 1
Data dictionary and source catalogue	Markdown / XLSX	Week 2
EDA notebook and findings log	Jupyter / HTML	Week 2
Preprocessing and feature pipeline	Python modules	Week 3
Experiment tracking log	MLflow	Week 4
Model evaluation and selection report	DOCX / PDF	Week 5
Deployment and monitoring plan	Markdown	Week 6
Final presentation	PPTX	Week 6

Table 8.1 — Deliverables schedule.

8.2 Repository Structure

The repository is organised so that the pipeline can be reproduced end to end from raw data with a single command, and so that notebooks are used for exploration only, never as the home of production logic.

```

churn-prediction/
├── data/
│   ├── raw/           # immutable source files, never modified
│   ├── interim/       # cleaned intermediate outputs
│   └── processed/     # final feature matrices
├── notebooks/        # exploration only, numbered in order
├── src/
│   ├── data/         # ingestion and schema validation
│   ├── features/      # feature engineering transformers
│   ├── models/        # training, tuning, prediction
│   └── evaluation/    # metrics, cost analysis, fairness audit
├── configs/          # YAML parameters and schema contracts
├── models/           # serialised model artefacts
├── reports/          # figures and written outputs
├── tests/            # unit tests for pipeline components
├── requirements.txt   # pinned dependencies
└── README.md

```

Figure 8.1 — Planned repository structure.

9. Conclusion

This blueprint defines a churn prediction project end to end before any modelling work begins. It frames a specific business problem — moving retention from reactive to proactive — and translates it into a supervised binary classification task with measurable, business-anchored success criteria. It specifies the data required, identifies four public datasets that collectively cover that specification, and is candid about where those datasets fall short.

The pipeline design prioritises correctness over sophistication. Structural controls against leakage, evaluation metrics chosen for an imbalanced problem, and a threshold derived from a cost matrix rather than assumed all reflect a single principle: the most common way a churn model fails is not that the algorithm was too weak, but that the evaluation was measuring the wrong thing. The algorithm comparison follows the same logic, with a decision rule set in advance that favours the simpler model unless the complex one earns its additional cost.

The six-week, 168-hour timeline allocates the majority of effort to data work rather than modelling, and the risk register identifies leakage and class imbalance as the two issues most likely to determine the outcome. With the gates at each milestone, the project has defined points at which it can be corrected rather than only a defined point at which it ends.

10. References

- IBM (2019). Telco Customer Churn Dataset. IBM Community / Kaggle.
- Duke University Teradata Center. Cell2Cell Telecom Churn Dataset.
- KDD Cup 2009. Customer Relationship Prediction Challenge (Orange Telecom).
- Dua, D. and Graff, C. UCI Machine Learning Repository. University of California, Irvine.
- Pedregosa et al. (2011). Scikit-learn: Machine Learning in Python. JMLR 12, pp. 2825–2830.
- Chen, T. and Guestrin, C. (2016). XGBoost: A Scalable Tree Boosting System. KDD '16.
- Lundberg, S. and Lee, S. (2017). A Unified Approach to Interpreting Model Predictions. NeurIPS.
- Chawla et al. (2002). SMOTE: Synthetic Minority Over-sampling Technique. JAIR 16, pp. 321–357.
- Saito, T. and Rehmsmeier, M. (2015). The Precision-Recall Plot Is More Informative than the ROC Plot on Imbalanced Datasets. PLOS ONE.
- Kaufman, S., Rosset, S. and Perlich, C. (2012). Leakage in Data Mining: Formulation, Detection, and Avoidance. ACM TKDD.